

University of Armed Forces - ESPE

Ostin Torres

Josselyn Ulcuango

Mateo Unda

Yandry Vélez

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Object Oriented Programming

PhD Edison Lascano

Project: LegalSync — Legal Case Management System

NRC 36862

October 26, 2026

Quito - Sangolquí

Version: 2.0 — **Date:** October 26, 2026

Document Version History

Version	Date	Authors	Description of Changes
1.0	October 17, 2026	Team CodeLex	Initial Software Requirements Specification (IEEE 830).
2.0	October 26, 2026	Team CodeLex	Full alignment with client elicitation and course technical features: console navigable menu via keyboard, JSON data persistence, CSV report generation, 6-tier email-based role hierarchy managed by the Managing Director, active/passive case classification by legal category, chronological calendar schedule, and itemized case expense/debt tracking.

1. INTRODUCTION

1.1. Purpose

The purpose of this document is to specify the functional and non-functional requirements of LegalSync, an object-oriented software solution designed to manage legal cases, client profiles, procedural tasks, and client debt records for a law firm. This document establishes the system's required functionalities, the six user profiles that will interact with it, the technical constraints it must respect (keyboard input/output via a navigable console menu, JSON data persistence, and CSV report generation), and the criteria used to verify compliance with its requirements. This specification serves as the primary reference for the analysis, object-oriented design, clean code implementation, and unit testing phases of the project.

1.2. Product Scope

LegalSync centralizes information related to system users, clients, defendants, active and passive court cases, assigned lawyers, judicial units, document references, procedural deadlines, and itemized legal expenses. Currently, the law firm performs case and billing management using manual spreadsheets and separate folders, leading to duplicated work, difficulties locating case files, lack of role restrictions, and problems tracking how much a client owes or which user modified a docket.

The system allows authorized users to authenticate via email credentials, manage a 6-tier staff hierarchy, register and classify cases (by Case ID, docket number, legal branch, process class, priority, and active/passive state), search records using multiple criteria, log lawyer dispositions and consult a chronological schedule calendar, track itemized case expenses and client debts, attach document file paths, audit system modifications, and export structured data reports in CSV format while persisting all system entities in JSON files.

1.3. Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification (IEEE 830 standard).
LegalSync	Name of the Legal Case Management System.
CodeLex	Name of the project development team.
CaseFile	Core entity representing a judicial docket, including classification, parties, tasks, documents, and expenses.

Managing Director	Principal lawyer (*Director Principal*) at the top of the firm's hierarchy with full privileges to enable accounts and assign roles by email.
Secretary	Administrative profile restricted strictly to read-only visualization of trial statuses, the schedule calendar, and client billing/expense records.
Expense	An itemized procedural or legal cost recorded within a case file (with amount, date, description, and payment status) used to calculate client debt.
JSON	JavaScript Object Notation format used by the system for persistent data storage.
CSV	Comma-Separated Values format used by the system to generate structured data and financial reports.
RBAC	Role-Based Access Control.
FR / NFR	Functional Requirement / Non-Functional Requirement.

1.4. Project References

This specification is based on the following project artifacts:

1. **HW04_Definition (v2.0):** Problem definition, system overview, and background of the LegalSync system.
2. **HW05_Requirement (v2.0):** Product Backlog (PB-001 through PB-009), including priorities and acceptance criteria.
3. **List of Possible Objects:** Candidate domain, service, and persistence objects identified for the system.

4. **UML Diagrams (UCD LegalSync and Class Diagram):** Use case and class diagrams modeling the 6 actors, entities, and relationships.

1.5. Document Conventions

Requirements are identified using unique codes to ensure full traceability:

- **FR-XXX:** Functional Requirement (mapped directly to the 9 mandatory operations in HW05).
- **NFR-XXX:** Non-Functional Requirement.
- **UC-XXX:** System Use Case.
- **BR-XXX:** Business Rule.
- **AC-XXX:** Acceptance Criterion.

2. OVERALL DESCRIPTION

2.1. Product Perspective

LegalSync replaces scattered spreadsheets and manual tracking with a centralized object-oriented console application written in Java. The system operates through an interactive, navigable console menu using keyboard input and output. All data entities generated by the system are stored persistently in local **JSON** files (users.json, clients.json, cases.json, and audit_log.json), while summary and financial statements are exported as **CSV** data reports (cases_report.csv and billing_report.csv). Its modular architecture separates domain models, business services, persistence utilities, and console views so the core logic can evolve in future iterations.

2.2. Problem Description

Manual legal case management through spreadsheets presents the following critical issues:

1. **Lack of hierarchical access control:** Six different staff profiles interact with records without email-based credential validation or role restrictions.
2. **Poor case classification and slow retrieval:** Staff cannot rapidly separate active from passive cases or filter dockets by Case ID, client name, judicial code, priority, or legal category (Criminal, Family/Alimony, Civil, Labor, Transit).
3. **Untracked client expenses and debts:** There is no itemized ledger inside each case to record specific legal costs (e.g., "$5.00 for notarized copies") with descriptions, making it difficult to know how much a client owes or whether their balance is paid in full.
4. **Disorganized dispositions and court dates:** Lawyers lack a unified schedule calendar and task tracker to assign dispositions to Junior Lawyers or Legal Assistants by Judicial Unit.

5. **Absence of modification traceability and automated reporting:** Spreadsheets do not log which user modified a docket or generate standardized CSV reports for administrative and billing review.

2.3. System Objectives

2.3.1. General Objective

To develop an object-oriented legal case management system operated via a navigable console menu that centralizes clients, categorized case files, email-based role permissions, chronological schedule consultation, itemized client debt tracking, JSON data persistence, and CSV report generation.

2.3.2. Specific Objectives

- Authenticate users via email and masked password, enforcing a 6-tier role hierarchy controlled by the Managing Director.
- Register and update clients, defendants, and categorized legal case files (ACTIVE / PASSIVE) validating mandatory fields.
- Provide a multi-criteria search engine by Case ID, Client Name, Legal Category, and Case Status.
- Record itemized procedural expenses per case and automatically calculate outstanding client debts.
- Manage lawyer dispositions, pending tasks, and a chronological court schedule calendar.
- Link local document and evidence file paths to case records.
- Persist all system data in JSON files, track user modifications in an audit log, and generate CSV data reports.

2.4. User Classes and Characteristics

Based on the law firm's organizational structure, the system defines **six user roles** managed through the User class using a String role attribute:

Role / Actor	**Description and Responsibilities**
Managing Director (DIRECTOR)	Principal authority at the top of the firm's hierarchy. Holds full system access; exclusively enables/disables user accounts, reviews users,

	and assigns or modifies roles linked to staff emails.
Partner (PARTNER)	Senior lawyer with full operational access to register/update cases, issue dispositions, record case expenses, and generate reports.
Associate Lawyer (ASSOCIATE_LAWYER)	Manages assigned cases, registers procedural dispositions, updates case statuses, links documents/evidence, and logs legal expenses.
Junior Lawyer (JUNIOR_LAWYER)	Assists in case management, consults the schedule calendar, updates assigned pending tasks, and registers evidence/document paths.
Legal Assistant (LEGAL_ASSISTANT)	Assigned to a specific Judicial Unit (*Unidad Judicial*); updates procedural task statuses, links document references, and consults case data.
Secretary (SECRETARY)	Strictly restricted to **read-only visualization** of trial statuses, the chronological schedule calendar, client debts, and billing/CSV reports. Cannot modify case files or credentials.

2.5. Operating Environment

- **Platform & Language:** Java SE (JDK) developed in Apache NetBeans IDE.
- **User Interaction:** Standard console input/output via keyboard using a navigable numeric menu.
- **Data Persistence:** Local **JSON** files (users.json, clients.json, cases.json, audit_log.json) automatically read and written by the system.

- **Reporting Output:** Local **CSV** files (cases_report.csv, billing_report.csv) compatible with spreadsheet viewers.

2.6. Product Constraints

1. All system interactions shall be performed via keyboard input and console output through a navigable menu.
2. All persistent data generated by the system shall be stored in **JSON** files.
3. All system reports shall be exported in **CSV** format.
4. At least 9 complete operations shall be included and 100% implemented in the source code.
5. Only the DIRECTOR role shall have privileges to enable user accounts and modify role assignments by email.
6. The SECRETARY role shall be blocked from executing any creation, update, or deletion operation on case files.
7. Document and evidence attachments shall be stored as local file path text references without parsing internal binary content.

2.7. Assumptions and Dependencies

- Staff members have unique email addresses registered by the Managing Director.
- JSON serialization and deserialization rely on standard Java file I/O and JSON processing libraries in the project classpath.
- Users enter dates in the standard YYYY-MM-DD (LocalDate) format validated by exception handling.

3. FUNCTIONAL REQUIREMENTS (THE 9 MANDATORY OPERATIONS)

The functional requirements correspond directly to the **9 operations** defined in the Product Backlog (HW05_Requirement v2.0) and implemented in the console menu.

3.1. FR-001: Hierarchical Role & Email Access Management

- **Priority:** High | **Source:** PB-001.
- **Description:** The system shall enforce Role-Based Access Control (RBAC) across the 6 staff roles, allowing the Managing Director to create accounts, enable/disable access, review users, and assign roles by email.
- **Specific Requirements:**

- **FR-001.1.** The system shall support six roles: DIRECTOR, PARTNER, ASSOCIATE_LAWYER, JUNIOR_LAWYER, LEGAL_ASSISTANT, and SECRETARY.
- **FR-001.2.** Only a logged-in user with the DIRECTOR role shall be able to register new users, change a user's assigned role by email, or enable/disable user accounts.
- **FR-001.3.** The system shall restrict the SECRETARY role exclusively to read-only consultation of trial statuses, the schedule calendar, and expense/billing records.
- **FR-001.4.** All user account and role changes shall be persisted immediately in users.json.

3.2. FR-002: Credential Security & Authentication (Login / Logout)

- **Priority:** High | **Source:** PB-002.
- **Description:** The system shall validate user access via registered email and password before displaying the role-specific console menu.
- **Specific Requirements:**
  - **FR-002.1.** The system shall request the user's email and password via keyboard input upon startup.
  - **FR-002.2.** The system shall verify credentials against users.json and check that the user's active status is true.
  - **FR-002.3.** Passwords shall be masked or handled securely, and users shall be able to change their password and log out to return to the authentication prompt.

3.3. FR-003: Client and Defendant Registration & Management

- **Priority:** High | **Source:** PB-003.
- **Description:** The system shall allow authorized users to register, view, and update client profiles and defendant records via the console.
- **Specific Requirements:**
  - **FR-003.1.** Each client record shall store clientId, idNumber (Cédula/RUC), name, phone, email, and address.
  - **FR-003.2.** Each defendant record shall store defendantId, idNumber, name, contactInfo, and address.
  - **FR-003.3.** The system shall validate mandatory fields before saving and persist client records in clients.json.

3.4. FR-004: Case Registration, Coding & Legal Classification

- **Priority:** High | **Source:** PB-003 / PB-004.

- **Description:** The system shall allow authorized staff to register and update legal case files with complete judicial classification and state tracking.
- **Specific Requirements:**
    - **FR-004.1.** Each case file shall include mandatory fields: unique caseId, official docketNumber (*codificación de juicio*), title, processClass (*clase de proceso*), category (CRIMINAL, FAMILY, CIVIL, LABOR, TRANSIT), priority (HIGH, MEDIUM, LOW), judicialUnit, status (ACTIVE, PASSIVE, IN_PROGRESS, SUSPENDED, CLOSED), openingDate, client, defendant, and assignedLawyer.
    - **FR-004.2.** Authorized users (excluding SECRETARY) shall be able to update the procedural state of a case (e.g., transitioning a case from ACTIVE to PASSIVE/CLOSED).
    - **FR-004.3.** All created and updated case files shall be saved persistently in cases.json.

3.5. FR-005: Multi-Criteria Case & Client Search Engine

- **Priority:** High | **Source:** PB-004.
- **Description:** The system shall allow users to rapidly search and filter case files across in-memory collections loaded from JSON using multiple criteria.
- **Specific Requirements:**
    - **FR-005.1.** The search menu shall support filtering cases by: (a) **Case ID**, (b) **Client Name**, (c) **Legal Category** (CRIMINAL, FAMILY, CIVIL, LABOR, TRANSIT), and (d) **Case Status** (ACTIVE vs. PASSIVE).
    - **FR-005.2.** Selecting a retrieved case shall display its complete details, the associated client's contact profile, and a summary of the client's debt/payment status.

3.6. FR-006: Case Expense & Client Debt Ledger

- **Priority:** High | **Source:** PB-005.
- **Description:** The system shall record itemized procedural expenses within each case file and automatically calculate the client's total outstanding debt.

- **Specific Requirements:**
    - **FR-006.1.** Authorized users shall be able to add an Expense object to a case file specifying expenseId, amount (e.g., $5.00), description (e.g., "Notarized copies for alimony filing"), expenseDate, and paid status (true/false).
    - **FR-006.2.** Authorized users shall be able to mark a pending expense item as paid.

- **FR-006.3.** The system shall automatically compute the total expenses, total paid amount, and remaining debt balance for the case and client, displaying whether the account is PENDING or PAID IN FULL (*Completo*).
- **FR-006.4.** All expense updates shall be saved inside cases.json.

3.7. FR-007: Lawyer Dispositions, Pending Tasks & Schedule Calendar

- **Priority:** High | **Source:** PB-006.
- **Description:** The system shall allow lawyers to register procedural dispositions and tasks with target dates and display a chronologically ordered court schedule calendar in the console.
- **Specific Requirements:**
  - **FR-007.1.** Lawyers shall be able to add a Deadline / task to a case including deadlineId, description, disposition (*disposición del abogado*), type (Hearing, Filing, Task), targetDate, assignedUser (Junior Lawyer or Legal Assistant by Judicial Unit), and completed status.
  - **FR-007.2.** Authorized staff shall be able to modify pending tasks and mark them as completed.
  - **FR-007.3.** The system shall calculate the remaining calendar days (Target Date - Current Date) and display a chronologically ordered schedule calendar in the console for all roles, including the SECRETARY.

3.8. FR-008: Document & Evidence Reference Registration

- **Priority:** Medium | **Source:** PB-007.
- **Description:** The system shall link local file paths of legal documents and evidence to specific case files.
- **Specific Requirements:**
  - **FR-008.1.** Authorized users shall be able to attach a Document (documentId, name, type, filePath, uploadDate) or Evidence (evidenceId, description, type, filePath, collectionDate) to a case file.
  - **FR-008.2.** The file path (e.g., "C:\cases\alimony_claim.pdf") shall be stored as a text attribute within cases.json without parsing the binary file content.

3.9. FR-009: CSV Data Reports Generation & Audit Log Consultation

- **Priority:** Medium | **Source:** PB-008 / PB-009.
- **Description:** The system shall automatically record user modifications in a JSON audit log and export structured case and billing/debt reports in CSV format.

- **Specific Requirements:**
    - **FR-009.1.** Every creation or modification of a user role, case file, pending task, or expense record shall append an AuditEntry (dateTime, userEmail, role, action, docketNumber) to audit_log.json.
    - **FR-009.2.** The Managing Director and Partners shall be able to view the chronological audit history in the console.
    - **FR-009.3.** The system shall generate two CSV reports upon request: cases_report.csv (listing all cases with ID, docket code, client, category, priority, judicial unit, and active/passive status) and billing_report.csv (listing clients, cases, itemized expenses, total billed, and pending debt for Secretary and Director review).

4. NON-FUNCTIONAL REQUIREMENTS

- **NFR-001 (Console Usability & Navigation):** The user interface shall operate entirely via a clean, numbered, navigable console menu with keyboard input validation and clear feedback messages.
- **NFR-002 (JSON Data Persistence):** All domain records (User, Client, CaseFile, AuditEntry) shall be automatically persisted to and loaded from structured .json files generated by the system.
- **NFR-003 (CSV Report Interoperability):** Generated .csv reports shall use standard comma delimiters and headers so they can be opened directly in Microsoft Excel without formatting errors.
- **NFR-004 (Security & Role Enforcement):** Every menu operation shall verify the logged-in user's Role before execution, strictly enforcing the Managing Director's exclusive privileges and the Secretary's read-only restrictions.
- **NFR-005 (Exception Handling & Robustness):** Invalid keyboard inputs (such as non-numeric menu selections, malformed dates, or negative expense amounts) and file I/O errors shall be caught using standard and custom Java exceptions without crashing the program.
- **NFR-006 (Clean Code & Object-Oriented Design):** The source code shall follow Clean Code standards, strict encapsulation (private attributes with constructors, getters, and setters), package separation, and the four UML class relationships (Composition, Aggregation, Association, and Dependency)

5. EXTERNAL INTERFACE REQUIREMENTS

5.1. User Interface (INT-001)

The system provides a text-based console interface navigated via keyboard input:

1. **Authentication Screen:** Prompts for registered email and password.
2. **Main Navigable Console Menu:** Displays numbered options (1 to 9, plus 0 to Log Out) adapted to the logged-in user's role.
3. **Formatted Console Tables:** Displays search results, the chronological schedule calendar, audit logs, and itemized client debt statements in aligned text tables.

5.2. Local File Persistence & Reporting Interface (INT-002)

File Name	Format	Direction	Purpose
data/users.json	JSON	Read / Write	Persistent storage of the 6-tier staff accounts, emails, passwords, and roles.
data/clients.json	JSON	Read / Write	Persistent storage of registered clients and defendants.
data/cases.json	JSON	Read / Write	Persistent storage of case files, categories, dispositions/deadlines, document paths, and itemized expenses.
data/audit_log.json	JSON	Read / Write	Chronological audit log of all modifications performed by users.

reports/cases_report.csv	CSV	Output (Report)	Tabular report of active and passive cases classified by legal branch and priority.
reports/billing_report.csv	CSV	Output (Report)	Financial report detailing case expenses, paid amounts, and pending client debts.

6. DATA MODEL AND MAIN ENTITIES

This section details the classes modeled in the UML Class Diagram (Class Diagram v2.0):

- **User :**
  - *Attributes:* -userId : int, -name : String, -email : String, -password : String, -role : String , -active : boolean.
  - *Operations:* +login(email : String, password : String) : boolean, +logout() : void, +changePassword(oldPass : String, newPass : String) : boolean, +hasPermission(permission : String) : boolean.
- **Lawyer:**
  - *Attributes:* -position : String, -licenseNumber : String.
  - *Operations:* +isDirector() : boolean, +assignUserRole(user : User, newRole : Role) : boolean, +restrictUserAccess(user : User) : void, +getDetails() : String.
- **LegalAssistant:**
  - *Attributes:* -judicialUnit : String.
  - *Operations:* +updateCaseData(caseFile : CaseFile) : boolean.
- **Secretary :**
  - *Attributes:* -officeArea : String.
  - *Operations:* +viewCaseStatus(caseFile : CaseFile) : String, +viewScheduleCalendar(deadlines : List<Deadline>) : void, +viewCaseExpenses(caseFile : CaseFile) : List<Expense>.
- **CaseFile:**
  - *Attributes:* -caseId : int, -docketNumber : String, -title : String, -processClass : String, -category: String , -priority: String , -judicialUnit : String,-status: String , -openingDate : LocalDate, -client : Client, -defendant : Defendant, -assignedLawyer :

Lawyer, -documents : List<Document>, -evidences : List<Evidence>, -deadlines : List<Deadline>, -expenses : List<Expense>.

    - *Operations:* +validateMandatoryFields() : boolean, +addDocument(document : Document) : void, +addEvidence(evidence : Evidence) : void, +addDeadline(deadline : Deadline) : void, +addExpense(expense : Expense) : void, +calculateTotalDebt() : double, +isPaidInFull() : boolean.

- **Expense (Composed within CaseFile):**
    - *Attributes:* -expenseId : int, -caseId : int, -description : String, -amount : double, -expenseDate : LocalDate, -paid : boolean.
    - *Operations:* +markAsPaid() : void, +getDetails() : String.
- **Deadline (Composed within CaseFile):**
    - *Attributes:* -deadlineId : int, -caseId : int, -description : String, -disposition : String, -type : String, -targetDate : LocalDate, -assignedUser : String, -completed : boolean.
    - *Operations:* +calculateRemainingDays() : int, +markCompleted() : void.
- **Client and Defendant:**
    - *Client Attributes:* -clientId : int, -idNumber : String, -name : String, -phone : String, -email : String, -address : String.
    - *Defendant Attributes:* -defendantId : int, -idNumber : String, -name : String, -contactInfo : String, -address : String.
- **Document and Evidence:**
    - Store metadata and local file path references (-filePath : String) associated with a CaseFile.
- **AuditLog and AuditEntry:**
    - Record and persist every modification event (-dateTime : LocalDateTime, -username : String, -action : String, -docketNumber : String) in audit_log.json.
- **SearchEngine, AuthenticationService, JsonFileManager, and CsvReportGenerator:**
    - Service and utility classes providing multi-criteria filtering, credential verification, JSON file persistence, and CSV report generation.

7. SYSTEM USE CASES

- **UC-001: Authenticate and Manage User Roles (FR-001, FR-002):** User logs in via keyboard with email and password. If the user is the DIRECTOR, they can create accounts, enable/disable access, and modify roles in users.json.
- **UC-002: Register and Update Clients & Defendants (FR-003):** Authorized staff enters client and defendant data via console and saves it to clients.json.

- **UC-003: Register and Classify Case File (FR-004):** Lawyer registers a new case with Case ID, docket number, legal category, process class, priority, judicial unit, and ACTIVE/PASSIVE state in cases.json.
- **UC-004: Update Case Status and Pending Actions (FR-004):** Authorized user updates trial status or modifies pending information, generating an automatic entry in audit_log.json.
- **UC-005: Search Cases by Multi-Criteria & Inspect Client Debt (FR-005):** Any user searches cases by ID, Client Name, Legal Category, or Active/Passive status and views the linked client profile and debt summary.
- **UC-006: Record Case Expenses and Update Payment Status (FR-006):** Authorized user logs a legal expense with amount and description inside a case or marks an item as paid, recalculating the client's total debt in cases.json.
- **UC-007: Register Dispositions and Consult Schedule Calendar (FR-007):** Lawyers add dispositions/tasks with target dates; staff and the Secretary consult the chronological schedule calendar in the console.
- **UC-008: Attach Document and Evidence File Paths (FR-008):** Authorized user links local PDF/Word file paths to a case file in cases.json.
- **UC-009: Export CSV Data Reports and View Audit Log (FR-009):** User generates cases_report.csv and billing_report.csv, or inspects the modification history stored in audit_log.json.

8. BUSINESS RULES

- **BR-001 (Director Exclusivity):** Only the Managing Director (DIRECTOR) can enable user accounts or change the role associated with a staff email address.
- **BR-002 (Secretary Read-Only Restriction):** Users with the SECRETARY role can view trial statuses, the schedule calendar, client debts, and CSV reports, but are strictly prohibited from creating or modifying cases, tasks, or users.
- **BR-003 (Mandatory Classification):** Every case file must have a unique ID, judicial docket code, legal category, priority level, and ACTIVE/PASSIVE status before being persisted to cases.json.
- **BR-004 (Debt Calculation Rule):** A client's case debt equals the sum of all Expense amounts where paid == false. A case financial balance is marked PAID IN FULL only when total unpaid expenses equal $0.00.
- **BR-005 (Remaining Days Calculation):** For any incomplete deadline or hearing, Remaining Days = Target Date - Current Date.
- **BR-006 (Mandatory Audit Trail):** Every state-changing operation on cases, expenses, tasks, or roles must append an entry to audit_log.json.

9. REQUIREMENTS TRACEABILITY MATRIX AND ACCEPTANCE CRITERIA

Backlog ID	Console Menu Operation	SRS Requirement	Persistence / Output	Acceptance Criterion Code & Verification
PB-001	Op. 1: Manage Users & Roles (Director)	**FR-001**	users.json	**AC-001:** Director assigns roles by email; Secretary is blocked from write operations.
PB-002	Op. 2: Login / Change Password	**FR-002**	users.json	**AC-002:** Valid active credentials grant access; invalid or disabled accounts are rejected.
PB-003	Op. 3: Register/Update Clients & Defendants	**FR-003**	clients.json	**AC-003:** Client and defendant objects are validated and saved to JSON.

PB-004	Op. 4: Register & Classify Case Files	**FR-004**	cases.json	**AC-004:** Cases are saved with ID, docket code, legal category, priority, and Active/Passive state.
PB-005	Op. 5: Multi-Criteria Case & Client Search	**FR-005**	cases.json	**AC-005:** Search retrieves exact matches by ID, Client, Category, or Status and shows client debt.
PB-006	Op. 6: Case Expense & Client Debt Ledger	**FR-006**	cases.json	**AC-006:** Expenses are logged with descriptions and amounts; pending debt is calculated accurately.
PB-007	Op. 7: Dispositions, Tasks & Schedule Calendar	**FR-007**	cases.json	**AC-007:** Tasks/hearings are stored and displayed in chronological order with remaining days.

PB-008	Op. 8: Link Document/Evidence File Paths	**FR-008**	cases.json	**AC-008:** Local file paths are linked as text attributes to the selected case file.
PB-009	Op. 9: Generate CSV Reports & View Audit Log	**FR-009**	*.csv / audit_log.json	**AC-009:** System exports cases_report.csv and billing_report.csv and displays audit history.

10. IDENTIFIED RISKS AND MITIGATION

Code	Risk	Possible Consequence	Mitigation in Implementation
R-001	Invalid keyboard input (e.g., letters instead of numbers or invalid dates).	Application crash (InputMismatchException or DateTimeParseException).	Implement centralized keyboard input validation utilities with try-catch blocks.
R-002	Unauthorized role attempting to	Compromised legal and financial integrity.	Check currentUser.getRole() before every write operation and throw custom UnauthorizedActionException.

	modify case or billing data.		
R-003	Missing or corrupted local JSON files on first run.	Null pointer or file-not-found errors on startup.	Automatically initialize default directories (data/, reports/), empty JSON arrays, and a default DIRECTOR account if files do not exist.

11. VERIFICATION, VALIDATION, AND UNIT TESTING

In compliance with the course artifacts (Test cases and Unit tests):

1. **Unit Testing (JUnit):** Automated unit tests shall verify:
    - Accurate calculation of unpaid client debt (CaseFile.calculateTotalDebt()) and full payment status (CaseFile.isPaidInFull()).
    - Accurate calculation of remaining days for court hearings (Deadline.calculateRemainingDays()).
    - Multi-criteria search filtering by Case ID, Client Name, Legal Category, and Active/Passive status (SearchEngine).
    - Role permission checks ensuring DIRECTOR has full access and SECRETARY is read-only.
2. **Persistence & Reporting Validation:** Functional test cases shall verify that every created object is properly serialized into .json files and that cases_report.csv and billing_report.csv are generated with valid comma-separated records.

12. CONCLUSIONS

The **LegalSync (V2.0)** Software Requirements Specification establishes a complete, realistic, and verifiable blueprint that bridges the real-world operational needs of the law firm with the object-oriented programming standards of the course. By structuring the system around a navigable console menu, JSON data persistence, CSV report generation, a 6-tier email-based role hierarchy, multi-criteria legal classification, chronological calendar consultation, and an itemized client debt ledger, all nine backlog functionalities can be cleanly implemented, tested, and demonstrated without scope ambiguity.